In this book we've focused on the "application" part of figure 27.1—the ASP.NET Core application itself—but the reality is that sometimes you'll want to place your ASP.NET Core apps behind a reverse proxy, such as IIS in Windows or NGINX or Apache in Linux. The *reverse proxy* is the program that listens for HTTP requests from the internet and then makes requests to your app as though the request came from the internet directly.

**DEFINITION** A *reverse proxy* is software that's responsible for receiving requests and forwarding them to the appropriate web server. The reverse proxy is exposed directly to the internet, whereas the underlying web server is exposed only to the proxy.

If you're running your application using a Platform as a Service (PaaS) offering such as Azure App Service, you're using a reverse proxy there too—one that is managed by Azure. Using a reverse proxy has many benefits:

- *Security*—Reverse proxies are specifically designed to be exposed to malicious internet traffic, so they're typically extremely well-tested and battle-hardened.
- *Performance*—You can configure reverse proxies to provide performance improvements by aggressively caching responses to requests.
- *Process management*—An unfortunate reality is that apps sometimes crash. Some reverse proxies can act as monitors/schedulers to ensure that if an app crashes, the proxy can automatically restart it.
- *Support for multiple apps*—It's common to have multiple apps running on a single server. Using a reverse proxy makes it easier to support this scenario by using the host name of a request to decide which app should receive the request.